Phase 2 - AX.25 Bit / HDLC Layer

จาก logical frame body 55 bytes สู่ HDLC/NRZI bitstream - เห็นทั้งเส้นทางส่งและเส้นทางรับกลับ

1 Block Diagram - ข้อมูลไหลอย่างไร?

รับ frame body จาก Phase 1 → แปลงเป็นบิต → ป้องกัน flag collision → ใส่ HDLC boundary → เข้ารหัส NRZI → ส่งต่อ Physical/RF

Phase 1 AX.25 frame → **Frame Body** 55 bytes → **LSB-first** 440 bits → **Bit Stuffing** 441 bits → **HDLC Flags** 457 bits → **NRZI** 457 levels → **Physical / RF** Next integration

Receive / Decode Path: NRZI levels → NRZI decode → ตรวจ HDLC flags → de-stuff → bits-to-bytes → ได้ frame body 55 bytes เดิมกลับมา

INPUT - รับมาจากไหน?

รับ **AX.25 frame body 55 bytes** จาก Phase 1 โดยตัด logical opening/closing flag ระดับ byte ออกก่อน เพื่อไม่ให้ flag ถูกนำไป bit stuffing.

OUTPUT - ส่งต่อให้ใคร?

ได้ **NRZI level stream 457 levels** ที่พร้อมส่งต่อให้ modem / physical / RF chain. ตอนนี้ยังเป็น software-level 0/1 ไม่ใช่คลื่น RF จริง.

ขอบเขตของ Phase 2: ทำงานระดับ **Bit / HDLC / Line Coding**. ยังไม่ทำ modulation, symbol shaping, SDR hardware, Doppler หรือ RF link testing.

FRAME BODY	LSB-FIRST	STUFFED	HDLC + 2 FLAGS	NRZI
55 bytes	440 bits	441 bits	457 bits	457 levels

2 แต่ละ Block ทำอะไร และใช้ตอนไหน?

อ่านตาม data flow จาก Frame Body ไปถึง NRZI และย้อนกลับที่ receiver

1) Frame Body - 55 bytes

ทำอะไร: เป็นข้อมูล AX.25 จริงที่ต้องส่ง ได้แก่ address, control, PID, information และ FCS.

ใช้ตอนไหน: เป็น input ของ Phase 2 หลังถอด logical flags ของ Phase 1 ออก.

`57-byte frame → 55-byte body`

2) LSB-first Serialization

ทำอะไร: แปลงทุก byte เป็น 8 bits โดยส่ง bit ตำแหน่งต่ำสุดก่อน.

ใช้ตอนไหน: ก่อนทำ HDLC bit processing เพราะลำดับ bit บนสายส่งต้องชัดเจน.

ตัวอย่าง: `0x8E = 10001110 → 01110001`

3) Bit Stuffing

ทำอะไร: หลังเจอ **1 ติดต่อกัน 5 ตัว** ให้ insert 0 หนึ่งตัว.

ใช้ตอนไหน: ใช้เฉพาะข้อมูลภายใน frame เพื่อป้องกัน pattern ภายในไปเหมือน HDLC flag.

`440 bits → 441 bits` ใน test vector นี้เพิ่มจริง 1 bit.

4) HDLC Flags

ทำอะไร: ใส่ opening และ closing flag `01111110` อย่างละ 8 bits.

ใช้ตอนไหน: หลัง stuffing เสร็จ เพื่อให้ receiver รู้ขอบเขต frame.

`8 + 441 + 8 = 457 bits`

5) NRZI Encode / Decode

ทำอะไร: แปลง bit เป็นการเปลี่ยน/คงระดับสัญญาณ.

ใช้ตอนไหน: หลังประกอบ HDLC stream และก่อน physical/RF layer.

Convention: bit 0 = transition, bit 1 = no transition.

6) Reverse / Receive Path

ทำอะไร: ทำย้อนกลับ NRZI → HDLC → de-stuff → bits-to-bytes.

ใช้ตอนไหน: เมื่อ receiver ต้องกู้ frame body เดิมจาก bitstream ที่รับมา.

`Recovered body = 55 bytes` และต้องเท่าต้นฉบับ.

ทำไมลำดับต้องเป็นแบบนี้? ต้อง serialize ก่อนจึงจะตรวจ run ของบิตได้; stuffing ต้องเกิดก่อนใส่ flag เพราะ flag ห้ามถูก stuffed; NRZI ต้องทำหลังได้ HDLC bitstream ที่สมบูรณ์แล้ว.

Bytes → **LSB-first** → **Stuff** → **Add Flags** → **NRZI**

สิ่งที่ยังไม่ควรสรุป: PASS ถึง NRZI ไม่ได้แปลว่า RF link พร้อมใช้งาน. Modulation, symbol rate, filtering, SDR integration และ channel effects ยังอยู่นอก Phase 2.

3 Code - Logic สำคัญอยู่ตรงไหน?

โค้ดจริงที่เปลี่ยน frame bytes เป็น HDLC/NRZI และย้อนกลับได้

A) Byte → Bits (LSB-first)

```
def byte_to_bits_lsb_first(value: int) -> list[int]:
    if not 0 <= value <= 0xFF:
        raise ValueError("Value must be between 0x00 and
0xFF")
    bits = []
    for bit_position in range(8):
        bit = (value >> bit_position) & 0x01
        bits.append(bit)
    return bits
```

B) Bit Stuffing / De-stuffing

```
def bit_stuff(bits):
    out, ones = [], 0
    for bit in bits:
        out.append(bit)
        ones = ones + 1 if bit == 1 else 0
        if ones == 5:
            out.append(0)
            ones = 0
    return out

def bit_destuff(bits):
    # เมื่อเห็น 1 ครบ 5 ตัว ต้องคาดหวัง stuffed 0
    # แล้วลบเฉพาะ 0 ที่ encoder เพิ่มเข้าไป
    ...
```

C) Build / Decode HDLC Stream

```
HDLC_FLAG = 0x7E

def build_hdlc_bitstream(frame_body):
    flag_bits = byte_to_bits_lsb_first(HDLC_FLAG)
    body_bits = bytes_to_bits_lsb_first(frame_body)
    stuffed = bit_stuff(body_bits)
    return flag_bits + stuffed + flag_bits

def decode_hdlc_bitstream(hdlc_bits):
    flag_bits = byte_to_bits_lsb_first(HDLC_FLAG)
    if hdlc_bits[:8] != flag_bits:
        raise ValueError("Missing opening HDLC flag")
    if hdlc_bits[-8:] != flag_bits:
        raise ValueError("Missing closing HDLC flag")
    frame_bits = bit_destuff(hdlc_bits[8:-8])
    return bits_to_bytes_lsb_first(frame_bits)
```

D) NRZI Encode / Decode

```
def nrzi_encode(bits, initial_level=0):
    level = initial_level
    levels = []
    for bit in bits:
        if bit == 0:
            level ^= 1    # 0 = transition
        levels.append(level) # 1 = no transition
    return levels

def nrzi_decode(levels, initial_level=0):
    previous = initial_level
    bits = []
    for level in levels:
        bits.append(1 if level == previous else 0)
        previous = level
    return bits
```

1 **`(value >> bit_position) & 1`**

ดึง bit จากตำแหน่ง 0 → 7 จึงได้ลำดับ LSB-first โดยตรง เช่น `0x8E` → `01110001`.

2 **Stuff หลัง 1 ครบ 5 ตัว**

ทันทีที่ counter ถึง 5 ให้เพิ่ม 0 แล้ว reset counter เพื่อไม่ให้ข้อมูลภายในสร้าง pattern ใกล้เคียง flag.

3 **Flags อยู่นอก stuffing**

`flag + stuffed + flag` ทำให้ flag จริงยังคงเป็น `01111110` และ receiver หา boundary ได้.

4 **Decoder ตรวจ flag ก่อน de-stuff**

หากไม่มี opening/closing flag ที่ถูกต้อง ให้หยุด ไม่แปลง stream ที่ขอบเขตไม่สมบูรณ์.

5 **`level ^= 1`**

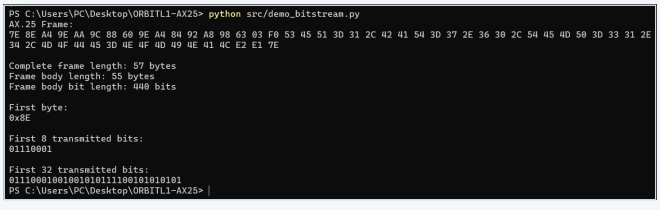
เมื่อ input bit เป็น 0 ระดับสัญญาณจะ toggle. ถ้าเป็น 1 ระดับเดิมถูกเก็บไว้.

6 **Round trip เป็นหลักฐานสำคัญ**

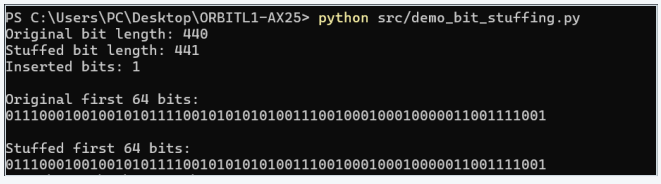
ถ้า encode → decode แล้วได้ bitstream/body เดิม แสดงว่า transformation แต่ละชั้นย้อนกลับได้โดยไม่ทำข้อมูลเสีย.

4 Test Evidence - เราพิสูจน์อะไรได้แล้ว?

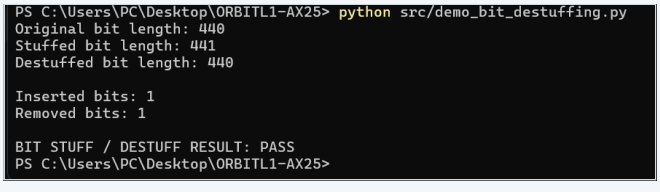
ใช้ทั้ง terminal evidence จริงและ round-trip checks เพื่อปิด Phase 2



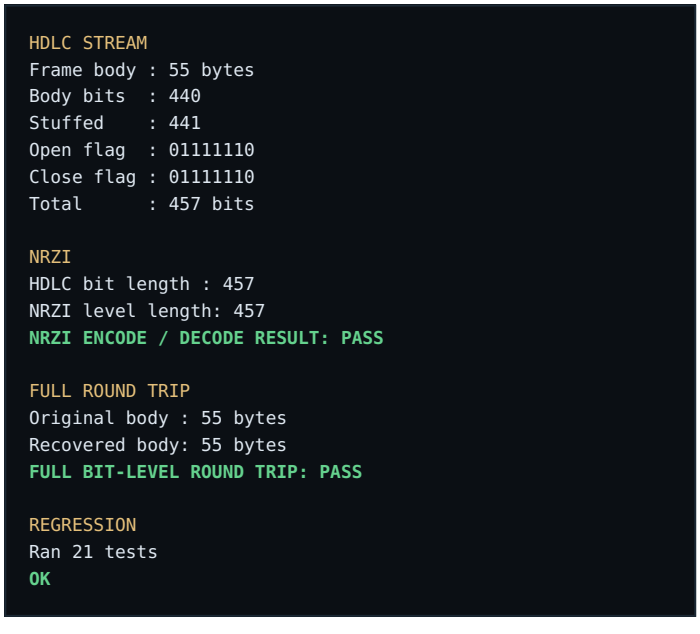
Serialization Test: Complete frame 57 bytes → frame body 55 bytes → 440 bits; byte แรก 0x8E ถูกส่งเป็น `01110001`.



Bit Stuffing Test: 440 → 441 bits และมี inserted bit = 1 ตาม test vector นี้.



De-stuffing Test: 441 → 440 bits, inserted=1, removed=1 และผล **PASS**.



440	441	457	21/21
original bits 55 bytes × 8	after stuffing +1 inserted bit	HDLC bits / NRZI levels + 2 flags	combined automated tests Phase 1 + Phase 2

- ✓ LSB-first order ตรงตามที่คาด
- ✓ HDLC stream length = 457 bits
- ✓ NRZI encode/decode ได้ bits เดิม
- ✓ Phase 2 functions ไม่ทำ Phase 1 tests พัง
- ✓ Stuff / de-stuff round trip ได้
- ✓ Opening/closing flag = 01111110
- ✓ Recovered frame body = original 55 bytes
- ✓ รวม regression 21 tests / OK

Phase 2 Exit: Software chain ตั้งแต่ frame body → bits → stuffing → HDLC → NRZI → decode กลับ ผ่านแล้ว. งานถัดไปคือ **Phase 3 - ORBITL-1 Packet Mapping** เพื่อกำหนด mission packet ที่จะใส่ใน AX.25 Information Field.